Library Management System

1️⃣ What this project should prove

By the end, this app should demonstrate that you can,

- Think in **objects**, not scripts
- Model a **real-world system** using classes
- Apply **control flow**, **collections**, and **LINQ**
- Handle **user input safely**
- Structure a program cleanly (not everything in Main)
- Make reasonable design decisions

This is not about UI. It's about **logic and structure**.

2️⃣ Core Features (Must-Have)

These are **non-negotiable** for a solid LMS console app.

📖 Book Management

- Add a book
- View all books
- Search books
    - by title
    - by author
    - by ISBN
- Update book details
- Remove a book

Book properties - Id, Title, Author, ISBN, IsAvailable

👥 Member Management

- Add a library member
- View all members
- Search members
- Remove a member

Member properties - Id, Name, Email, Phone

🔄 Borrow & Return System

- Borrow a book
  - Only if available
  - Linked to a member
- Return a book
- Track borrowed books

Rules

- A book can’t be borrowed twice
- A member can’t borrow unavailable books

3️⃣ Project Structure (Very Important)

```
LibraryManagementSystem/
│
├── Models/
│   ├── Book.cs
│   ├── Member.cs
│   ├── Loan.cs
│
├── Services/
│   ├── LibraryService.cs
│
├── Utils/
│   ├── InputHelper.cs
│
└── Program.cs
```

This shows **professional thinking**, even in a console app.

4️⃣ Core Classes & Responsibilities

📘 Book

- Holds book data
- No business logic

👤 Member

- Holds member data
- No borrowing logic inside

🔁 Loan

- Connects a Book + Member
- Has:
    - BorrowDate
    - ReturnDate (nullable)

🧠 LibraryService (Brain of the app)

This is where **most logic lives**.

Responsibilities:

- Store collections:
    - List<Book>
    - List<Member>
    - List<Loan>
- Add / remove entities
- Borrow & return logic
- Search logic

5️⃣ Console Menu Design

Use a **loop-based menu**,

==== Library Management System ====

1. Manage Books

2. Manage Members

3. Borrow Book

4. Return Book

5. View Reports

0. Exit

Each option calls a **method**, not inline code.

6️⃣ Concepts You'll Cover (Mapping to Your Course)

This project naturally covers:

C# Concept	Where you use it
Classes & Objects	Book, Member, Loan
Encapsulation	Private lists in services
Collections	List<T>
LINQ	Search, filtering
Conditionals	Borrow rules
Loops	Menus
Enums	Menu choices, statuses
Nullable types	ReturnDate
Exception handling	Invalid input
Methods	Clean separation of logic

7️⃣ Stretch Features (Highly Recommended)

Once core is done, add these 👇

🔍 Reports

- Currently borrowed books
- Overdue books (if you add due dates)
- Books borrowed by a member

💾 Data Persistence

Start simple:

- Save/load data from **JSON file**
- Use System.Text.Json

This will massively level up the project.

🧪 Validation & Errors

- Prevent empty inputs
- Handle invalid menu choices
- Graceful error messages (no crashes)

8️⃣ How to Build This (Step-by-Step Plan)

Do NOT code everything at once.

Phase 1

- Create models (Book, Member)
- Hardcode sample data
- Display lists

Phase 2

- Add menus
- Add CRUD operations

Phase 3

- Borrow & return logic

Phase 4

- Refactor (clean methods, folders)

Phase 5

- Persistence (JSON)
- Reports